


# Autonomous Vehicle Simulation (AVS) Laboratory, University of Colorado

## Basilisk Technical Memorandum

**Document ID: Basilisk-avsLibrary**

### AVS LIBRARY OF ASTRODYNAMICS AND C-BASED LINEAR ALGEBRA SUPPORT SUB-ROUTINES


| | |
|---|---|
| Prepared by | H. Schaub |


**Status:** Revised

**Scope/Contents**


The AVS Library has released a series of C, Matlab and Mathematica support routines to perform common astrodynamics functions. This document outlines how these support files are automatically validated within the Basilisk environment. There are C-based functions that run each routine with sample input, and compare to hand computed answers if the function response is correct. This checking is performed automatically every time `pytest` is run on the root Basilisk folder.


| Rev | Change Description | By | Date |
|---|---|---|---|
| 1.0 | Initial Revision | H. Schaub | 08/2017 |
| 1.1 | Small typo fixes | H. Schaub | 10/18/2017 |
| 1.2 | Orbital-motion numerical methods, equations, API compatibility, and regression coverage (issue 1629) | AVS Lab | 10/18/2026 |

# Contents


**1 Library Description** **2**
- 1.1 `linearAlgebra` Library . . . 2
- 1.2 `rigidBodyKinematics` Library . . . 2
- 1.3 `orbitalMotion` Library . . . 2
  - 1.3.1 Classical Orbital Elements . . . 2
  - 1.3.2 Orbit Element to Cartesian Coordinate Conversions . . . 4
    - 1.3.2.1 `elem2rv()` Function . . . 4
    - 1.3.2.2 `rv2elem()` Function . . . 5
  - 1.3.3 Space Environment Parameters . . . 7
    - 1.3.3.1 Atmospheric Density . . . 7
    - 1.3.3.2 Mean Debye Length . . . 7
    - 1.3.3.3 Atmospheric Drag Acceleration . . . 7
    - 1.3.3.4 Gravitational Zonal Harmonics . . . 7
    - 1.3.3.5 Solar Radiation Disturbance Acceleration . . . 8

**2 Library Functions** **8**

**3 Library Assumptions and Limitations** **9**
- 3.1 Assumptions . . . 9
  - 3.1.1 Linear Algebra Library . . . 9
  - 3.1.2 Rigid Body Kinematics . . . 9
  - 3.1.3 Orbital Motion . . . 9
- 3.2 Limitations . . . 9
  - 3.2.1 Linear Algebra Library . . . 9
  - 3.2.2 Rigid Body Kinematics . . . 9
  - 3.2.3 Orbital Motion . . . 9

**4 Test Description and Success Criteria** **10**

**5 Test Parameters** **10**
- 5.1 Linear Algebra . . . 10
- 5.2 Rigid Body Kinematics . . . 10
- 5.3 Orbital Motion . . . 10

**6 Test Results** **10**

**7 User Guide** **11**
- 7.1 `linearAlgebra` Library . . . 11
  - 7.1.1 Vector Operations . . . 11
  - 7.1.2 Matrix Operations . . . 12
- 7.2 `rigidBodyKinematics` Library . . . 13
- 7.3 `orbitalMotion` Library . . . 13
  - 7.3.1 Orbit Anomaly Angle Conversions . . . 13
  - 7.3.2 Orbit Element Conversion . . . 14
    - 7.3.2.1 Mean/Osculating and Equinoctial Domains . . . 14
    - 7.3.2.2 Hill Frame Domain . . . 15

7.3.2.3 Compatibility . . . 15
7.3.3 Space Environment Functions . . . 15
7.3.3.1 Atmospheric Density . . . 15
7.3.3.2 Mean Debye Length . . . 15
7.3.3.3 Atmospheric Drag Acceleration . . . 15
7.3.3.4 Gravitational Zonal Harmonics . . . 15
7.3.3.5 Solar Radiation Pressure Acceleration . . . 16

# 1 Library Description

The AVS lab has released a set of astrodynamics C-functions* that facilitate common orbital or rotational dynamics evaluations. The functions are openly available, and are included with the AIAA Education Series textbook *Analytical Mechanics of Space Systems*[1] since the first release in 2003. The most up to date public versions are found on the AVS web page.

## 1.1 `linearAlgebra` Library

The linear algebra library provides numerous C-based functions to perform basic matrix math operations. For a complete list of functions supported, consult the `linearAlgebra.h` file. The library provides common matrix and vector manipulation tools such as performing a matrix product, including the transpose operators, as well as doing the matrix inverse operations. The vector library supports doing dot and cross product operations, as well as taking a norm of a vector. Helper functions are provided to initial zero or identity matrices, as well as check if the matrix elements are zero

## 1.2 `rigidBodyKinematics` Library

The rigid body kinematics library provides a range of functions related to mapping between attitude coordinates descriptions, compute the addition or subtraction of orientations, as well as returning the differential kinematic equations. This library has been included with Reference 1 since 2003 and has been used extensively across a range of research projects. Reference 1 provides all the mathematical details of the transformation used here in Chapter 3, while Appendix E provides a comprehensive discussion of all the algorithms included.

## 1.3 `orbitalMotion` Library

Common celestial mechanics tools are implemented in the `orbitalMotion` library, including transforming between anomaly angles, converting between inertial Cartesian position and velocity vector components and classical orbit elements, as well as evaluating simple space environment parameters. The following developments use the notation and formulation of Chapter 9 in Reference 1. The variable naming is reflected within the software code.

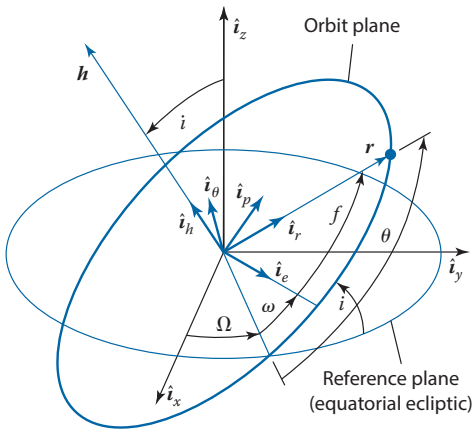
### 1.3.1 Classical Orbital Elements

The classical orbit elements are given by

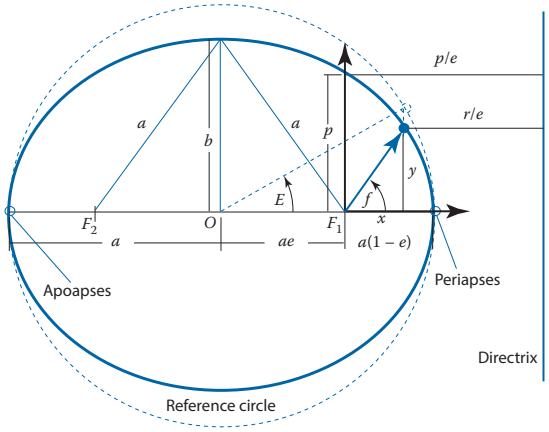
$$(a, e, i, \Omega, \omega, f)$$

where $a$ is the semi-major axis or SMA, $e$ is the eccentricity, and $(\Omega, i, \omega)$ are the 3-1-3 Euler angles called the right ascension of the ascending node, inclination, and argument of periapsis, respectively.

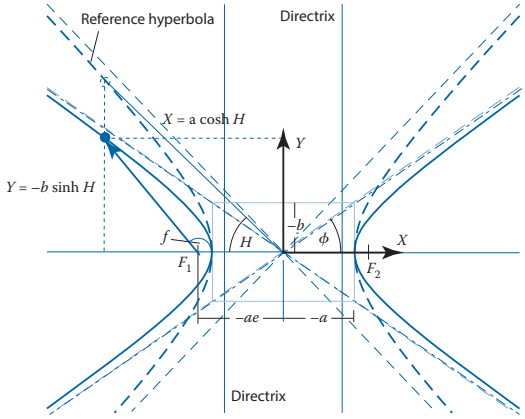
* http://hanspeterschaub.info/AVS-Code.html




(a) Orbit Frame Orientation Illustration




(b) Classical orbital parameter Illustration for an Elliptic Orbit




(c) Classical orbital parameter Illustration for a Hyperbolic Orbit

**Fig. 1:** Orbit Elements, Axes and Frames Illustrations.[1]

These are illustrated in Figure 1(a). Inclination satisfies $0 \leqslant i \leqslant \pi$. For nonparabolic motion the semi-latus rectum $p$ is

$$p = a(1 - e^2) \tag{1}$$

Finally, the true anomaly angle is given by $f$, while the eccentric anomaly angle is expressed through $E$ as illustrated in Figure 1(b) for an elliptic orbit scenario. The true and eccentric anomaly are related through[1]

$$\tan\left(\frac{f}{2}\right) = \sqrt{\frac{1+e}{1-e}}\tan\left(\frac{E}{2}\right) \tag{2}$$

The mean anomaly angle $M$ relates to the eccentric anomaly angle $E$ through[1]

$$M = E - e\sin E \tag{3}$$

The classical orbit elements for a hyperbolic scenario are shown in Figure 1(c). The SMA is negative for a hyperbolic orbit. For nonrectilinear motion, the true anomaly $f$ measures position from periapsis. Rectilinear motion instead uses an energy-dependent anomaly convention described below. The

hyperbolic anomaly $H$ relates to $f$ through

$$\tan\left(\frac{f}{2}\right) = \sqrt{\frac{e+1}{e-1}}\tanh\left(\frac{H}{2}\right) \tag{4}$$

The hyperbolic mean anomaly is defined in terms of the hyperbolic anomaly through

$$N = e\sinh H - H \tag{5}$$

The inverse mean-anomaly mappings solve Kepler's equation with bracketed Newton iteration, using bisection whenever a Newton step leaves the bracket. Stable residual and derivative expressions reduce cancellation near unit eccentricity and avoid overflow for large hyperbolic anomalies. Iteration stops when the correction is at most $4\epsilon$ times the magnitude of the current anomaly, or when the next representable iterate is unchanged; $\epsilon$ is double-precision machine epsilon. The limit is 200 iterations. Invalid inputs or exhausted iteration produce a diagnostic and NaN in C, or an exception in Python. Elliptic results retain whole revolutions, subject to the resolution of the input and output angles.

### 1.3.2 Orbit Element to Cartesian Coordinate Conversions

Let ${}^{\mathcal{N}}\boldsymbol{r}_{C/N}$ and ${}^{\mathcal{N}}\boldsymbol{v}_{C/N}$ be the inertial spacecraft center of mass $C$ position and velocity vector matrix representations, expressed with respect to inertial frame $\mathcal{N}:\{\hat{\boldsymbol{n}}_1,\hat{\boldsymbol{n}}_2,\hat{\boldsymbol{n}}_3\}$ components. Functions are provided to map from classical orbit elements to these Cartesian coordinates through `elem2rv()`, as well as the inverse mapping from Cartesian to classical orbit elements through `rv2elem()`.

**1.3.2.1 `elem2rv()` Function** The general conversion of classical orbit elements to inertial Cartesian components is outlined in section 9.6.2 of Reference 1. For nonrectilinear motion, the orbit radius at true anomaly $f$ is

$$r = \frac{p}{1+e\cos f} \tag{6}$$

If the satellite is not on a parabola, then $p$ is computed using

$$p = a(1-e^2) \tag{7}$$

For a nonrectilinear parabola, the core C and Python conversions use the sentinel $a = 0$, $e = 1$, and

$$p = 2r_p \tag{8}$$

where $r_p > 0$ is supplied in `rPeriap`. Eccentricity $e = 1$ alone does not identify a parabola, because rectilinear motion also has unit eccentricity. Distances must use the same unit L as the gravitational parameter $\mu$ in $\mathrm{L}^3/\mathrm{s}^2$; angles are in radians. Open-conic anomalies must satisfy $|f| < \arccos(-1/e)$ after reduction to the principal interval. Collisions and anomalies at or beyond infinity are rejected. Trigonometric evaluation retains the phase of large wrapped inputs without division by a rounded $2\pi$.

The true latitude angle $\theta$ is given by

$$\theta = \omega + f \tag{9}$$

The inertial position vector is then given by[1]

$$\boldsymbol{r}_{C/N} = r\begin{pmatrix}\cos\Omega\cos\theta - \sin\Omega\sin\theta\cos i\\ \sin\Omega\cos\theta + \cos\Omega\sin\theta\cos i\\ \sin\theta\sin i\end{pmatrix}^{\mathcal{N}} \tag{10}$$

while the inertial velocity vector is

$$\dot{\boldsymbol{r}}_{C/N} = -\frac{\mu}{h}\begin{pmatrix}\cos\Omega(\sin\theta + e\sin\omega) + \sin\Omega(\cos\theta + e\cos\omega)\cos i\\ \sin\Omega(\sin\theta + e\sin\omega) - \cos\Omega(\cos\theta + e\cos\omega)\cos i\\ -(\cos\theta + e\cos\omega)\sin i\end{pmatrix}^{\mathcal{N}} \tag{11}$$

where $\mu$ is the gravitational parameter and $h = |\boldsymbol{h}|$ is the magnitude of the specific orbital angular momentum $\boldsymbol{h} = \boldsymbol{r} \times \dot{\boldsymbol{r}}$.

For rectilinear motion with nonzero energy, `elem2rv()` uses $e = 1$ and nonzero $a$. The `f` field holds eccentric anomaly $E$ for $a > 0$, or signed hyperbolic anomaly $H$ for $a < 0$. The corresponding radii are[1]

$$r = a(1 - \cos E) = 2a \sin^2(E/2), \qquad a > 0 \tag{12}$$

and

$$r = a(1 - \cosh H) = -2a \sinh^2(H/2), \qquad a < 0. \tag{13}$$

For rectilinear parabolic motion, use $e = 1$, $a = 0$, `rPeriap`=0, and a positive radius in `rmag`. The `f` field is a direction flag: zero for outward motion, $\pi$ for inward motion. The radial unit direction vector for all rectilinear cases is

$${}^{\mathcal{N}}\hat{\boldsymbol{\imath}}_r = \begin{pmatrix} \cos\Omega\cos\omega - \sin\Omega\sin\omega\cos i \\ \sin\Omega\cos\omega + \cos\Omega\sin\omega\cos i \\ \sin\omega\sin i \end{pmatrix}^{\mathcal{N}} \tag{14}$$

The inertial position vector is then given by

$${}^{\mathcal{N}}\boldsymbol{r}_{C/N} = r\ {}^{\mathcal{N}}\hat{\boldsymbol{\imath}}_r \tag{15}$$

The inertial velocity is ${}^{\mathcal{N}}\boldsymbol{v}_{C/N} = v_r {}^{\mathcal{N}}\hat{\boldsymbol{\imath}}_r$, with signed radial speed

$$v_r = \begin{cases} \sqrt{\mu/a}\cot(E/2) & a > 0, \\ \sqrt{\mu/(-a)}\coth(H/2) & a < 0, \\ \sqrt{2\mu/r} & a = 0,\ f = 0, \\ -\sqrt{2\mu/r} & a = 0,\ f = \pi. \end{cases} \tag{16}$$

Thus positive $\sin E$ or positive $H$ selects outward motion in the core conversion. The plane about the radial ray is arbitrary. Python's `elem2rv_parab` retains its original inward-positive elliptic convention and interprets every $e = 1$, $a < 0$ input as a planar parabola with $r_p = -a$, regardless of auxiliary fields. Use the core `elem2rv` for rectilinear hyperbolas. Both conventions remain supported, and conversions leave input element storage unchanged.

**1.3.2.2 `rv2elem()` Function** The following equations describe the inverse mapping for nonrectilinear motion. The implementation scales velocity by $\sqrt{\mu/r}$ and uses normalized directions to avoid unit-dependent intermediate overflow. First the semi-latus rectum is computed using

$$p = \frac{\boldsymbol{h} \cdot \boldsymbol{h}}{\mu} \tag{17}$$

The line of nodes vector $\boldsymbol{n}$ is

$$\boldsymbol{n} = \hat{\boldsymbol{n}}_3 \times \boldsymbol{h} \tag{18}$$

Let $\hat{\boldsymbol{n}}$ be the normalized line of node axis. If $|\boldsymbol{n}|$ is near zero, then the orbit is equatorial and the line of nodes is ill-defined. In this case we set $\hat{\boldsymbol{n}} = \hat{\boldsymbol{n}}_1$ such that $\Omega = 0$.

The eccentricity vector $\boldsymbol{e}$ is computed using[1]

$$\boldsymbol{e} = \frac{\dot{\boldsymbol{r}} \times \boldsymbol{h}}{\mu} - \frac{\boldsymbol{r}}{r} \tag{19}$$

The eccentricity is then simply

$$e = |\boldsymbol{e}| \tag{20}$$

Let $\hat{\boldsymbol{i}}_e$ be the normalized eccentricity vector. If $e$ is near zero, then the orbit is near circular and the periapses direction is ill defined. In this case we set $\hat{\boldsymbol{i}}_e = \hat{\boldsymbol{n}}$ which results in $\omega = 0$.

Within this function the orbit radius is set through

$$r = |\boldsymbol{r}_{C/N}| \tag{21}$$

and the radius at periapses is set to

$$r_p = \frac{p}{1+e} \tag{22}$$

The SMA is computed using a robust method where first the inverse of the SMA, called $\alpha$, is evaluated using[1]

$$\alpha = \frac{2}{r} - \frac{v^2}{\mu} \tag{23}$$

If $\alpha$ is resolved as nonzero, $a$ is determined through

$$a = \frac{1}{\alpha} \tag{24}$$

while the radius at apoapses is set to

$$r_a = a(1+e). \tag{25}$$

For hyperbolic motion this is a signed algebraic value, not a physical apoapsis. Energy is treated as zero when $|2 - v^2 r/\mu| \leqslant 8\epsilon \max(2, v^2 r/\mu)$. In that parabolic case the core conversion sets

$$a = 0.0$$

and returns $r_a = 0$. Python's default `rv2elem` retains $a = -r_p$, `rApoap`$=-1$ for planar parabolas; `canonical=True` explicitly selects the core encoding. Near zero energy, relative accuracy of $a$ and reconstruction from rounded $a, e$ degrade with conditioning. Finite-energy nonradial states whose eccentricity rounds to one or the wrong side of one cannot be represented consistently and are rejected.

Next the orbit frame orientation angles $\Omega$, $i$ and $\omega$ must be determined. As classical elements are used, care must be given for particular singular orientations where some angles are ill-defined. First, assume a non-circular non-equatorial orbit scenario. In this case $\Omega$ is the angle between $\hat{\boldsymbol{n}}_1$ and $\boldsymbol{n}$ where care must be taken that the right quadrant is used.

$$\tan\Omega = \frac{(\hat{\boldsymbol{n}}_1 \times \hat{\boldsymbol{n}}) \cdot \hat{\boldsymbol{n}}_3}{\hat{\boldsymbol{n}}_1 \cdot \hat{\boldsymbol{n}}} \tag{26}$$

The argument of periapses is the angle between $\hat{\boldsymbol{n}}$ and $\boldsymbol{e}$, where again quadrants must be checked.

$$\tan\omega = \frac{(\hat{\boldsymbol{i}}_n \times \hat{\boldsymbol{i}}_e) \cdot \hat{\boldsymbol{i}}_h}{\hat{\boldsymbol{i}}_n \cdot \hat{\boldsymbol{i}}_e} \tag{27}$$

Finally, the inclination angle $i$ is the angle between $\boldsymbol{h}$ and $\hat{\boldsymbol{n}}_3$, but here no quadrants must be checked as this angle is defined as $0 \leqslant i \leqslant \pi$.

$$\cos i = \hat{\boldsymbol{i}}_h \cdot \hat{\boldsymbol{n}}_3 \tag{28}$$

The true anomaly is the angle from $\boldsymbol{e}$ to $\boldsymbol{r}$, again with quadrant checking:

$$\tan f = \frac{(\hat{\boldsymbol{i}}_e \times \hat{\boldsymbol{i}}_r) \cdot \hat{\boldsymbol{i}}_h}{\hat{\boldsymbol{i}}_e \cdot \hat{\boldsymbol{i}}_r} \tag{29}$$

These ratios are evaluated with `atan2` to preserve quadrants. C returns $\Omega$, $\omega$ and nonradial $f$ in $[0, 2\pi)$. Python's main inverse retains signed hyperbolic true anomaly; its `rv2elem_parab` adapter retains signed node, periapsis and nonradial anomaly angles.

The 3-1-3 Euler angles representing the orbit frame orientation have singular configurations. The following list discusses how each case is handled:

- **Equatorial non-circular orbit:** Set $\Omega = 0$ and $i = 0$ or $\pi$. Measure periapsis from the inertial x axis in the direction of orbital motion, including retrograde motion.
- **Inclined circular orbit:** Set $\omega = 0$ and let $f$ measure argument of latitude from the node.
- **Equatorial circular orbit:** Set $\Omega = \omega = 0$ and measure $f$ from the inertial x axis in the direction of orbital motion.

The singularity threshold is dimensionless: $e \leqslant 10^{-12}$ or $\sin i \leqslant 10^{-12}$. Replacing unresolved directions introduces relative state errors of that order. The legacy Python `rv2elem_parab` circular convention instead places periapsis at the current position and returns $f = 0$.

Rectilinear motion is detected when transverse velocity is within $8\epsilon$ of the local speed. It returns $e = 1$, $r_p = 0$, an arbitrary but deterministic plane about the outward radial ray, and the energy-dependent anomaly described above. In particular, zero angular momentum alone does not imply zero orbital energy.

### 1.3.3 Space Environment Parameters

**1.3.3.1 Atmospheric Density** The Earth density helper uses a sixth-order polynomial fit to $\log_{10}\rho$, based on Standard Atmosphere 1976 data for altitudes from 100 to 1000 km. It extrapolates below that range. Above 1000 km it retains the exponential tail $\log_{10}(\rho/[\mathrm{kg/m^3}]) = -7\times10^{-5}(h/[\mathrm{km}]) - 14.464$; density is not cut off at 1000 km.

**1.3.3.2 Mean Debye Length** The Earth Debye length model accepts altitudes from 200 to 35000 km and returns length in meters. Values above 1000 km are highly speculative.

**1.3.3.3 Atmospheric Drag Acceleration** The Earth drag helper opposes the supplied inertial velocity; it does not subtract an atmosphere rotation velocity. In consistent SI units the acceleration is

$$\boldsymbol{a}_d = -\frac{\rho C_d A}{2m}|\boldsymbol{v}|\boldsymbol{v}. \tag{30}$$

Here $\rho$ is density in kg/m$^3$, $C_d$ is dimensionless, $A$ is projected area in m$^2$, and $m$ is mass in kg. The API accepts position in km and velocity in km/s and converts internally to return km/s$^2$. Positive altitude above Earth's reference radius, positive mass, nonnegative $C_d, A$, and finite inputs are required. Valid zero speed, area, or coefficient returns zero acceleration. Invalid inputs or unrepresentable output produce NaNs with a diagnostic in C, or `ValueError` in Python. The density extrapolations above remain active; they do not extend the fit's validated altitude range.

**1.3.3.4 Gravitational Zonal Harmonics** The zonal helper sums Earth's contributions from $J_2$ through the requested degree, at most $J_6$. Other supported oblate bodies use $J_2$ only. These are perturbations without central gravity or tesseral terms, in a planet-centered frame whose z axis is aligned with the spin axis. Distances are in km and accelerations in km/s$^2$. The external expansion is intended for positions outside the body's reference radius; the API does not enforce that surface boundary. Planet selection and legacy zero-zonal behavior are described in the user guide. The individual contributions are[1]

$$\boldsymbol{a}_{J_2} = -\frac{3}{2}J_2\left(\frac{\mu}{r^2}\right)\left(\frac{r_{eq}}{r}\right)^{\mathcal{N}}\begin{pmatrix}\left(1-5\left(\frac{z}{r}\right)^2\right)\frac{x}{r}\\ \left(1-5\left(\frac{z}{r}\right)^2\right)\frac{y}{r}\\ \left(3-5\left(\frac{z}{r}\right)^2\right)\frac{z}{r}\end{pmatrix} \tag{31}$$

$$\boldsymbol{a}_{J_3} = \frac{1}{2} J_3 \left(\frac{\mu}{r^2}\right)\left(\frac{r_{eq}}{r}\right)^3 \mathcal{N}\begin{pmatrix} 5\left(7\left(\frac{z}{r}\right)^3 - 3\left(\frac{z}{r}\right)\right)\frac{x}{r} \\ 5\left(7\left(\frac{z}{r}\right)^3 - 3\left(\frac{z}{r}\right)\right)\frac{y}{r} \\ -3\left(10\left(\frac{z}{r}\right)^2 - \frac{35}{3}\left(\frac{z}{r}\right)^4 - 1\right) \end{pmatrix} \tag{32}$$

$$\boldsymbol{a}_{J_4} = \frac{5}{8} J_4 \left(\frac{\mu}{r^2}\right)\left(\frac{r_{eq}}{r}\right)^4 \mathcal{N}\begin{pmatrix} \left(3 - 42\left(\frac{z}{r}\right)^2 + 63\left(\frac{z}{r}\right)^4\right)\frac{x}{r} \\ \left(3 - 42\left(\frac{z}{r}\right)^2 + 63\left(\frac{z}{r}\right)^4\right)\frac{y}{r} \\ \left(15 - 70\left(\frac{z}{r}\right)^2 + 63\left(\frac{z}{r}\right)^4\right)\frac{z}{r} \end{pmatrix} \tag{33}$$

$$\boldsymbol{a}_{J_5} = \frac{J_5}{8} \left(\frac{\mu}{r^2}\right)\left(\frac{r_{eq}}{r}\right)^5 \mathcal{N}\begin{pmatrix} 3\left(35\left(\frac{z}{r}\right) - 210\left(\frac{z}{r}\right)^3 + 231\left(\frac{z}{r}\right)^5\right)\frac{x}{r} \\ 3\left(35\left(\frac{z}{r}\right) - 210\left(\frac{z}{r}\right)^3 + 231\left(\frac{z}{r}\right)^5\right)\frac{y}{r} \\ -\left(15 - 315\left(\frac{z}{r}\right)^2 + 945\left(\frac{z}{r}\right)^4 - 693\left(\frac{z}{r}\right)^6\right) \end{pmatrix} \tag{34}$$

$$\boldsymbol{a}_{J_6} = -\frac{J_6}{16} \left(\frac{\mu}{r^2}\right)\left(\frac{r_{eq}}{r}\right)^6 \mathcal{N}\begin{pmatrix} \left(35 - 945\left(\frac{z}{r}\right)^2 + 3465\left(\frac{z}{r}\right)^4 - 3003\left(\frac{z}{r}\right)^6\right)\frac{x}{r} \\ \left(35 - 945\left(\frac{z}{r}\right)^2 + 3465\left(\frac{z}{r}\right)^4 - 3003\left(\frac{z}{r}\right)^6\right)\frac{y}{r} \\ -\left(3003\left(\frac{z}{r}\right)^6 - 4851\left(\frac{z}{r}\right)^4 + 2205\left(\frac{z}{r}\right)^2 - 245\right)\frac{z}{r} \end{pmatrix} \tag{35}$$

**1.3.3.5 Solar Radiation Disturbance Acceleration** A simple solar disturbance acceleration $\boldsymbol{a}_d$ function is implemented where

$$\boldsymbol{a}_d = -\frac{C_r A \Pi}{mcs^3}\boldsymbol{s} \tag{36}$$

where $C_r = 1.3$ is the radiation pressure coefficient, $c = 299792458$m/s is the speed of light, $\Pi = 1361.0$ W/m$^2$ is the solar radiation flux and $\boldsymbol{s}$ is the sun position vector relative to the spacecraft in units of AU. The area $A$ is the projected area towards the sun.

# 2 Library Functions

The AVS astrodynamics support library is developed to facility writing C-code that must perform the associated mathematical functions. It is not a Basilisk module, but included with the main Basilisk distribution as a support library. The library goals are

- **C-Code implementation**: All function calls only use C-code.
- **Linear Algebra**: Basic matrix math function that are common in Matlab and Python environment quickly become tedious when programming in C. To avoid this, a custom written C library is developed to perform specialized 2, 3, 4, and 6 dimensional matrix math, but some linear algebra math on matrices of general size.
- **Rigid Body Kinematics**: Provides a comprehensive C-based software library to convert between a broad range of rigid body attitude coordinates, as well as add and subtract orientations. This library can also compute the differential kinematic equations of select coordinate types. The textbook Reference 1 contains a complete listing of all the attitude library function in Appendix E.
- **Orbital Motion Library**: Provides a comprehensive C-based software library to convert between orbit anomaly angles, as well as orbit elements and Cartesian coordinates. This library also contains some simple space environment modeling functions.

# 3 Library Assumptions and Limitations

## 3.1 Assumptions

### 3.1.1 Linear Algebra Library

The vector dimension can be declared either explicitly through

```
double vec[3]
```

or implicitly through

```
double *vec
```

and allocating the required memory dynamically. However, with the specialized matrix dimensions the matrices are assumed to be defined through commands like

```
double m33[3][3]
```

for a $3 \times 3$ matrix.

### 3.1.2 Rigid Body Kinematics

These attitude kinematic relationships are general enough that no assumptions are made on the mathematics. However, when the $3 \times 3$ DCM are defined, they must be of type

```
double BN[3][3]
```

### 3.1.3 Orbital Motion

This orbital motion library accepts the inertial position and velocity vectors as pointers to a 3-dimensional array. As with the linear algebra library, these can be declared explicitly or implicitly.

## 3.2 Limitations

### 3.2.1 Linear Algebra Library

There are no limitations on the provided linear algebra routines. They are written in a general manner.

### 3.2.2 Rigid Body Kinematics

There are no limitations on the provided linear algebra routines. They are written in a general manner.

### 3.2.3 Orbital Motion

The orbital motion support library has the following limitations.

- Both `elem2rv()` and `rv2elem()` support rectilinear states away from collision. The anomaly convention depends on energy. Hill-frame and elliptic-element mappings have narrower domains: optional `*Checked` entry points reject unsupported states, while the original names retain their historical fallback results and exceptions. All element-input APIs enforce inclination in $[0, \pi]$.
- The gravitational zonal harmonics for $J_3$-$J_6$ are only implemented for Earth, and not other celestial objects.
- The atmospheric density and drag, as well as the Debye length models are only implemented for Earth orbiting scenarios.

# 4 Test Description and Success Criteria

The AVS support libraries undergo a series of unit tests within C-functions that are called by the Swig'd interface `avsLibrarySelfCheck`. The automated unit test is performed via `pytest` through the script:

`architecture/utilitiesSelfCheck/_UnitTest/test_avsLibrarySelfCheck.py`

In each unit test the C-code library function output is compared to an externally hand-computed solution. The success criterion is when the pre-computed solution and the C-code computed solution match to within a certain numerical tolerance. The success criteria absolute value tolerances are shown in Table 2.

**Table 2:** Unit Test Absolute Difference Tolerances

| Test | Absolute Tolerance |
|---|---|
| Linear Algebra | $10^{-10}$ |
| Rigid Body Kinematics | $10^{-10}$ |
| Orbital Motion: Anomalies | $10^{-10}$ |
| Orbital Motion: Hill transformations | $10^{-4}$ |

# 5 Test Parameters

## 5.1 Linear Algebra

In these unit tests each function is provided non-trivial inputs, such as `v1 = [1,2,3]` to evaluate the associate matrix math. For specific information on the values used, look at the source code for the function `testLinearAlgebra()` in the file `avsLibrarySelfCheck.c`.

## 5.2 Rigid Body Kinematics

The attitude kinematics sub-routines are provided with non-trivial inputs to test the functions. For specific information on the values used, look at the source code for the function `testRigidBodyKinematics()` in the file `avsLibrarySelfCheck.c`.

## 5.3 Orbital Motion

The orbital anomaly sub-routines are provided with non-trivial inputs to test the functions. For specific information on the values used, look at the source code for the function `testOrbitalAnomalies()` in the file `avsLibrarySelfCheck.c`.

The Hill self-check, `testOrbitalHill()`, exercises rotating-frame state transformations. The native target `test_orbitalMotion` checks element conversions and environment models. Additional Python regressions are under `src/tests`, with integration tests in module test directories. These are separate from the C self-check entry points. Individual assertions define tolerances appropriate to each quantity and case. Compatibility regressions include repeated calls with Python and SWIG element containers, ignored auxiliary fields in the legacy parabolic adapter, and C compound-literal and C++ braced-initializer zonal calls. Large wrapped anomalies are checked against direct trigonometric state references in both languages.

# 6 Test Results

The four native self-check entry points exercised by this Python wrapper are listed in Table 3. Unknown selectors fail explicitly; no result is claimed for an entry point that does not exist.

**Table 3:** Integration test results.

| Test | Pass/Fail | BSK Error Notes |
|---|---|---|
| rigidBodyKinematics | PASSED | |
| orbitalHill | PASSED | |
| orbitalAnomalies | PASSED | |
| linearAlgebra | PASSED | |

# 7 User Guide

## 7.1 `linearAlgebra` Library

The linear algebra library provides numerous C-based functions to perform basic matrix math operations. For a complete list of functions supported, consult the `linearAlgebra.h` file.

### 7.1.1 Vector Operations

The vector related functions all begin with a letter 'v' and are broken down in the following categories depending on if the linear algebra operation is performed on a matrix of a general size, or of a specific size of 2, 3, 4 or 6.

- if the function is `vXXXX()` the code works on a $n$x1 matrix of arbitrary length $n$.
- if the function is `v2XXXX()` the code works on a matrix of length 2 defined as `double vec[2]`.
- if the function is `v3XXXX()` the code works on a matrix of length 2 defined as `double vec[3]`.
- if the function is `v4XXXX()` the code works on a matrix of length 2 defined as `double vec[4]`.

The most extensive set of linear algebra support functions are for operation on 3 and 4 dimensional matrices common with orbital mechanics (position and velocity vector representations) and attitude dynamics (working with 3- and 4-parameter attitude descriptions). The following list details what vector operation each function performs:

- `Copy`: Copies vector $\boldsymbol{v}_1$ into vector $\boldsymbol{v}_2$
- `SetZero`: Returns a zero vector
- `Set`: Sets the vector representation to specific values
- `Add`: Sums $\boldsymbol{v}_1$ and $\boldsymbol{v}_2$
- `Subtract`: Returns the difference $\boldsymbol{v}_1$ - $\boldsymbol{v}_2$
- `Scale`: Returns $\boldsymbol{v}_1$ multiplied by a scalar $\alpha$
- `Dot`: Returns the dot product $\boldsymbol{v}_1 \cdot \boldsymbol{v}_2$
- `OuterProduct`: Returns the outer product $\boldsymbol{v}_1 \boldsymbol{v}_2^T$
- `Normalize`: Returns a normalized vector $\boldsymbol{v}_1/v_1$
- `MaxAbs`: Returns the largest element of a vector
- `IsEqual`: Checks if two vector represnetations are identical
- `IsZero`: Checks if a vector is full of zero elements
- `Print`: Prints the vector representation to a file

- `Sort`: Sorts the vector elements by size
- `Print`: Prints the vector representation to a file
- `PrintScreen`: Prints the vector representation to the screen

### 7.1.2 Matrix Operations

The matrix related functions are all labeled with a letter 'M' and are broken down into the following matrix dimension related categories:

- `mXXXXy()`: the code works on a $n \times m$ matrix of arbitrary dimension $n$ and $m$.
- `m22XXXX()`: the code works on a $2 \times 2$ matrix defined through `mat[2][2]`.
- `m33XXXX()`: the code works on a $3 \times 3$ matrix defined through `mat[3]3]`.
- `m44XXXX()`: the code works on a $4 \times 4$ matrix defined through `mat[4][4]`.

The following list provides an overview of the supported matrix functions. Note that not al dimensions have all functions provided, but the 2, 3, and 4 dimensional matrix support is pretty complete.

- `SetIdentity`: Returns an identity matrix
- `SetZero`: Returns a zero matrix
- `Set`: Creates a matrix with specific values
- `Copy`: Copies the matrix $[M_1]$ into $[M_2]$
- `m33MultM33`: Performs the matrix to matrix multiplication $[M_1][M_2]$
- `m33MultM33t`: Performs the matrix to matrix multiplication $[M_1][M_2]^T$
- `m33tMultM33`: Performs the matrix to matrix multiplication $[M_1]^T[M_2]$
- `m33MultV3`: Computes $[M_1]\boldsymbol{v}_1$
- `m33tMultV3`: Computes $[M_1]^T\boldsymbol{v}_1$
- `v3tMultM33`: Computes $\boldsymbol{v}_1^T[M_1]$
- `v3tMultM33t`: Computes $\boldsymbol{v}_1^T[M_1]^T$
- `Tilde`: Returns the skew-symmetric matrix $[\tilde{\boldsymbol{v}}_1]$
- `Transpose`: Returns $[M_1]^T$
- `Add`: Returns the sum $[M_1] + [M_2]$
- `Subtract`: Returns the difference $[M_1] - [M_2]$
- `Scale`: Returns the scaled matrix $\alpha[M_1]$
- `Trace`: Compute the matrix trace $\sum M_{ii}$
- `Determinant`: Returns the square matrix determinant

- `IsEqual`: Checks if all matrix elements are equal
- `IsZero`: Checks if all matrix elements are zero
- `Print`: Prints the matrix representation to a file
- `PrintScreen`: Prints the matrix representation to the screen
- `Inverse`: Returns the matrix inverse $[M_1]^{-1}$
- `SingularValues`: Computes the singular values of $[M_1]$
- `EigenValues`: Computes the Eigenvalues of $[M_1]$
- `ConditionNumber`: Computes the condition number of $[M_1]$

## 7.2 `rigidBodyKinematics` Library

The following discussion is a brief overview of the `rigidBodyKinematics` library function notation. Please see Appendix E in Reference 1 for a complete description:

- `XXX2YYY`: Converts the attitude coordinates `XXX` to `YYY`
- `addXXX`: Add the two attitude descriptions $\boldsymbol{x}_1 = \boldsymbol{x}_{\mathcal{B}/\mathcal{F}}$ and $\boldsymbol{x}_2 = \boldsymbol{x}_{\mathcal{F}/\mathcal{N}}$ to return the sequential rotation of first $\boldsymbol{x}_1$ and then $\boldsymbol{x}_2$. Returns is the description $\boldsymbol{x}_3 = \boldsymbol{x}_{\mathcal{B}/\mathcal{N}}$
- `subXXX`: Subtract the two attitude descriptions $\boldsymbol{x}_1 = \boldsymbol{x}_{\mathcal{F}/\mathcal{N}}$ and $\boldsymbol{x}_2 = \boldsymbol{x}_{\mathcal{B}/\mathcal{N}}$ to return the relative orientation $\boldsymbol{x}_3 = \boldsymbol{x}_{\mathcal{F}/\mathcal{B}}$
- `BmatXXX`: Returns the matrix $[B]$ of the differential kinematic equations of the attitude parameters `XXX` in Reference 1. Note the scalar multiplier is not included here. For example, with MRPs[2] $[B]$ is defined as
$$\dot{\boldsymbol{\sigma}}_{\mathcal{B}/\mathcal{N}} = \frac{1}{4}[B(\boldsymbol{\sigma}_{\mathcal{B}/\mathcal{N}})]^{\mathcal{B}}\boldsymbol{\omega}_{\mathcal{B}/\mathcal{N}}$$
- `BinvXXX`: Returns the matrix inverse $[B]^{-1}$ of the attitude parameters `XXX`
- `dX`: Returns the time derivatives of the attitude parameters `XXX` as a function of these parameters and the body angular velocity vector $\boldsymbol{\omega}$
- `Mi`: Returns the single-axis DCM about the $\hat{\boldsymbol{b}}_i$ axis
- `wrapToPi`: Makes sure that an angle lies within $\pm\pi$

## 7.3 `orbitalMotion` Library

### 7.3.1 Orbit Anomaly Angle Conversions

To convert between the various anomaly angles, the following functions are defined:

- `double f2E(double f, double e)`
- `double E2f(double Ecc, double e)`
- `double E2M(double Ecc, double e)`
- `double M2E(double M, double e)`, using bracketed Newton iteration with a maximum of 200 iterations

- `double f2H(double f, double e)`
- `double H2f(double H, double e)`
- `double H2N(double H, double e)`
- `double N2H(double N, double e)`, using bracketed Newton iteration with a maximum of 200 iterations

Both solvers stop when the residual divided by the equation derivative is within four times machine epsilon of the anomaly magnitude, or when the root bracket cannot be refined at floating-point precision. Invalid inputs and iteration exhaustion produce an error diagnostic and return NaN.

### 7.3.2 Orbit Element Conversion

To convert from classical orbit elements to inertial Cartesian coordinates, the function

```
elem2rv(double mu, ClassicElements *elements, double *rVec, double *vVec)
```

is used where `mu` is the gravitational constant of the 2-body problem, the classical elements are defined through

$$(a, e, i, \Omega, \omega, f)$$

Inclination must lie in $0 \leqslant i \leqslant \pi$, including both equators. The anomaly is normally the true anomaly $f$. Eccentricity $e = 1$ alone does not identify a parabola: rectilinear states also have unit eccentricity. A nonrectilinear parabola uses $a = 0$, $e = 1$, and positive `rPeriap`. A rectilinear ellipse uses $a > 0$, $e = 1$, and eccentric anomaly $E$ in the `f` field; a rectilinear hyperbola uses $a < 0$ and hyperbolic anomaly $H$. A rectilinear parabola uses $a = 0$, $e = 1$, `rPeriap`=0, positive `rmag`, and `f`=0 for outward motion or $\pi$ for inward motion. The function returns the inertial position and velocity vectors in the arrays `rVec` and `vVec`.

To convert from inertial Cartesian coordinates to classical orbit elements, the function

```
rv2elem(double mu, double *rVec, double *vVec, ClassicElements *elements)
```

Beyond the classical elements listed above, this routine also stores periapsis and apoapsis radii $r_p$, $r_a$, and $\alpha = 1/a$. Python retains NumPy scalar fields and its default planar parabolic encoding $a = -r_p$, `rApoap`=$-1$. Optionally use `rv2elem(mu, r, v, canonical=True)` for the C-compatible encoding. The Python `*_parab` adapters preserve their signed angles and the inward-positive radial elliptic anomaly convention; negate $E$ when explicitly converting those inputs for the canonical `elem2rv` convention. Both conventions remain supported without deprecation warnings or a removal schedule. The forward adapter interprets every $e = 1$, $a < 0$ input as a planar parabola with $r_p = -a$, independently of auxiliary fields. Use the core `elem2rv` to reconstruct rectilinear hyperbolas. Python and SWIG input elements are not modified.

#### 7.3.2.1 Mean/Osculating and Equinoctial Domains

The checked first-order $J_2$ mapping requires finite elliptic inputs ($a > 0$, $0 \leqslant e < 1$, $0 \leqslant i \leqslant \pi$), positive equatorial radius, and direction $+1$ or $-1$. Nonzero $J_2$ rejects $|1 - 5\cos^2 i| \leqslant \sqrt{\epsilon}$, where $\epsilon$ is double-precision machine epsilon. Mapped elements must remain finite and elliptic; invalid results are not clipped into range. Zero $J_2$ copies the six valid input elements exactly, including at critical inclinations. These numerical guards do not establish perturbation accuracy: corrections must still be small. Use `clMeanOscMapChecked` for these checks.

The prograde equinoctial conversion `clElem2eqElemChecked` also requires finite elliptic inputs. It supports circular and prograde equatorial orbits but is singular at the retrograde equator, rejecting

$\cos(i/2) \leqslant 8\epsilon$. Coordinates close to that boundary remain ill-conditioned. Both routines emit diagnostics and all-NaN outputs on failure in C; their Python counterparts raise `ValueError` without changing the output object.

**7.3.2.2 Hill Frame Domain** The `hillFrameChecked`, `hill2rvChecked`, and `rv2hillChecked` routines require finite three-vectors and a nonzero chief position and velocity with a resolved orbital plane. They reject $\|\hat{\boldsymbol{r}} \times \hat{\boldsymbol{v}}\| \leqslant 8\epsilon$. Normalizing before crossing avoids dependence on distance units. State transformations additionally require a representable positive angular rate and finite outputs. Invalid inputs produce all-NaN outputs with a diagnostic in C or `ValueError` in Python. Distances must use consistent units, velocities the corresponding units per second.

**7.3.2.3 Compatibility** The original mapping and Hill function names use the improved calculation in its supported domain. Rejected domains retain historical formulas, exceptions, and sentinel outputs. Inclination bounds are always enforced. Original methods remain supported without deprecation warnings or a removal schedule; checked calls are optional additions.

### 7.3.3 Space Environment Functions

**7.3.3.1 Atmospheric Density** The Earth's atmospheric density $\rho$ is computed using

```
double atmosphericDensity(double alt)
```

The density is returned as a scalar value.

**7.3.3.2 Mean Debye Length** The mean Debye Length for the near-Earth environment is approximated, very crudely, through a polynomial fit. The function returns the scalar $\lambda_d$ and is called through

```
double debyeLength(double alt)
```

**7.3.3.3 Atmospheric Drag Acceleration** To compute an estimate of the Earth's atmospheric drag acceleration, use the function:

```
void atmosphericDrag(double Cd, double A, double m,
                     double *rvec, double *vvec, double *advec);
```

The inputs are the dimensionless drag coefficient `Cd`, projected area `A` in $m^2$, and spacecraft mass `m` in kg. The inertial position `rvec` is in km, velocity `vvec` in km/s, and returned acceleration `advec` in km/s$^2$. The atmosphere is assumed at rest in this inertial frame; no rotation or wind correction is applied. The density fit is valid from 100 to 1000 km; positive altitudes outside that range use the existing extrapolation, including an exponential tail above 1000 km rather than a hard cutoff.

All inputs and vector norms must be finite, with `Cd` and `A` nonnegative, `m` positive, and position above Earth's equatorial radius. Valid zero speed, area, coefficient, or computed density gives zero acceleration. Invalid inputs and acceleration overflow return three NaNs with a diagnostic in C and raise `ValueError` in Python. Output underflow is allowed.

**7.3.3.4 Gravitational Zonal Harmonics** The header safely supplies Earth for the three-argument C call:

```
jPerturb(rvec, num, ajtot);
```

Select another body with the typed interface:

```
void jPerturbForPlanet(double *rvec, int num, double *ajtot,
                       CelestialObject_t planetID);
jPerturbForPlanet(rvec, 2, ajtot, CELESTIAL_VENUS);
```

Four-argument `jPerturb` calls remain supported without deprecation warnings; the typed function above provides optional checked selection. The bare variadic symbol is retained for explicit-planet function pointers, which require a fourth integer argument. Three-argument pointers should target `jPerturbEarth`. Both source call forms accept C compound literals and C++ braced initializers. In Python, the zonal utility retains its string planet argument and defaults to Earth. Set `strict=True` to request checked selection.

Position is in km in a planet-centered frame whose $z$ axis is the spin axis; acceleration is returned in that frame in km/s$^2$. Earth supports degrees 2–6; other supported bodies provide $J_2$ only. Checked calls return zero for Sun and Pluto, while Phobos and Deimos are unsupported. The degree must be 2–6 and position must be finite and nonzero. Unsupported bodies, invalid inputs, and nonfinite acceleration produce a diagnostic and three NaNs in C, or `ValueError` in Python's checked path. The original planet selection retains Earth fallback for unknown bodies, unchanged native output for Sun/Pluto, and Python's `None` Sun/Pluto sentinel and floating degree cutoffs. These conventions remain supported.

**7.3.3.5 Solar Radiation Pressure Acceleration** To compute the inertial disturbance acceleration due to the solar radiation pressure use the function

```
void solarRad(double A, double m, double *sunvec, double *arvec)
```

Here `A` is the projected surface area, `m` is the spacecraft mass, `sunvec` is the sun position vector to the Sun in units of AU.

# REFERENCES

[1] Hanspeter Schaub and John L. Junkins. *Analytical Mechanics of Space Systems*. AIAA Education Series, Reston, VA, 3rd edition, 2014.

[2] Malcolm D. Shuster. A survey of attitude representations. *Journal of the Astronautical Sciences*, 41(4):439–517, 1993.